

Connector System Architecture

From Kernel to Business Operations

Connector Platform Team

March 22, 2026

Abstract

This document provides a comprehensive technical specification of the Connector platform architecture, spanning from low-level kernel operations through execution engines, protocol layers, developer experience surfaces, and business operations. Connector is an infrastructure-grade agentic runtime that provides governed, auditable, and recoverable execution for autonomous AI agents with memory-centric design, contract-based logic, and multi-protocol communication. This document covers: (1) the Memory Kernel and syscall interface, (2) execution engines including the Cognitive Substrate and CLS compiler/runtime, (3) protocol layers (CNP, A2A, MCP), (4) developer experience via the Glue facade and CLI, (5) business operations including signup, login, API key management, and license activation, and (6) local node operations including boot sequence, health monitoring, and agent deployment.

Contents

1	Introduction	2
1.1	System Overview	2
1.2	Design Principles	2
1.3	Target Deployment Models	2
1.4	Key Terminology	3
1.5	Architecture Diagram	3
1.6	Document Structure	4
2	Memory Kernel Layer	4
2.1	Architecture Overview	4
2.2	MemPacket Structure	4
2.2.1	Core Data Model	4
2.2.2	Content Addressing	5
2.3	Memory Kernel Syscalls	5
2.3.1	Memory Operations	5
2.3.2	Agent Lifecycle Operations	5
2.3.3	Session Management Operations	6
2.4	Agent Lifecycle State Machine	6
2.5	Session Management	6
2.5.1	Session Lifecycle	6
2.6	Predicate Enforcement	7
2.7	Cryptographic Integrity	7
2.7.1	Signing	7
2.7.2	Execution Receipts	7
2.8	Storage Backend	7

3	Execution Engines	8
3.1	Overview	8
3.2	Cognitive Substrate	8
3.2.1	11-Layer Thought Pipeline	8
3.2.2	Core Data Types	8
3.2.3	Expertise Kernels	9
3.2.4	Thought Checkpointing	9
3.3	CLS (Connector Logic Standard)	10
3.3.1	Contract Structure	10
3.3.2	Compiler Pipeline	10
3.3.3	Runtime Execution	10
3.3.4	State Machine Execution	11
3.4	Resource Budgets	11
3.5	Integration	11
4	Protocol Layers	11
4.1	Overview	11
4.2	CNP: Connector Native Protocol	12
4.2.1	Layer Architecture	12
4.2.2	Message Structure	12
4.2.3	Port-Based Communication	12
4.2.4	Capability Attenuation	13
4.2.5	Routing and Delivery	13
4.3	A2A: Agent-to-Agent Protocol	14
4.3.1	Message Format	14
4.3.2	A2A to CNP Bridge	14
4.4	MCP: Model Context Protocol	14
4.4.1	MCP Server Integration	15
4.4.2	Tool Invocation Flow	15
4.5	Protocol Selection Matrix	15
4.6	Security Properties	15
5	Developer Experience Layer	16
5.1	Design Philosophy	16
5.2	Glue Facade	16
5.2.1	Core Verbs	16
5.2.2	Nouns and Selectors	16
5.3	connectorctl CLI	17
5.3.1	Command Structure	17
5.3.2	Output Contracts	17
5.4	Glue API	18
5.4.1	Endpoint Structure	18
5.4.2	Response Format	18
5.5	Glue Rust Library	18
5.6	Error Handling	19
5.7	Configuration	19
5.8	Extensibility	20
6	Business Operations	20
6.1	Overview	20
6.2	Customer Signup Flow	20
6.2.1	Step-by-Step Process	20

6.2.2	Registration API	21
6.2.3	Database Schema	21
6.3	Login Flow	21
6.3.1	Portal Login (Email/Password)	21
6.3.2	Login API	22
6.3.3	Dashboard Login (Token-Only)	22
6.4	API Key Management	22
6.4.1	Key Issuance Flow	22
6.4.2	API Key Structure	23
6.4.3	Key Validation	23
6.5	License Activation	23
6.5.1	Activation Flow	23
6.5.2	License Structure	24
6.5.3	Activation API	24
6.6	Billing Integration	25
6.6.1	Stripe Integration	25
6.6.2	Webhook Handler	25
6.7	Tracking and Analytics	26
6.7.1	Usage Tracking	26
6.7.2	Metrics Collected	26
7	Node Operations	26
7.1	Overview	26
7.2	12-Stage Boot Sequence	27
7.2.1	Boot Reporter	27
7.2.2	systemd Integration	28
7.3	Health Probes	28
7.3.1	Liveness Probe	28
7.3.2	Readiness Probe	28
7.3.3	Startup Probe	29
7.4	Agent Deployment	29
7.4.1	Deployment Methods	29
7.4.2	Agent Manifest	29
7.4.3	Deployment Flow	30
7.5	Monitoring and Observability	30
7.5.1	Metrics Export	30
7.5.2	Structured Logging	31
7.5.3	Distributed Tracing	31
7.6	Graceful Shutdown	31
7.6.1	Shutdown Sequence	31
7.7	Backup and Restore	32
7.7.1	Backup	32
7.7.2	Restore	32
7.8	Upgrade Process	32
7.8.1	Rolling Upgrade	32
7.8.2	Rollback	33
7.9	Disaster Recovery	33
7.9.1	Recovery Scenarios	33
7.9.2	Recovery Time Objectives	33

1 Introduction

1.1 System Overview

Connector is a production-grade agentic runtime platform designed to execute autonomous AI agents with enterprise-level governance, auditability, and recovery guarantees. Unlike traditional agent frameworks that treat agents as stateless request handlers, Connector provides a **memory-centric kernel** where all agent state, communication, and computation are persisted as content-addressed memory packets with cryptographic integrity.

The platform is structured in seven architectural layers:

1. **Memory Kernel Layer** — Content-addressed storage, syscall interface, agent lifecycle management
2. **Execution Engine Layer** — Cognitive substrate, CLS compiler/runtime, contract execution
3. **Protocol Layer** — CNP (7-layer native protocol), A2A messaging, MCP bridges
4. **Logic Plane** — Contract registry, composition patterns, validation/certification
5. **Developer Experience Layer** — Glue facade, CLI grammar, SDK surfaces
6. **Business Operations Layer** — Signup, login, API keys, license activation, billing
7. **Infrastructure Layer** — Boot sequence, health probes, monitoring, deployment

1.2 Design Principles

- **Memory-First Architecture** — All state is memory; computation is memory transformation
- **Content Addressing** — Every artifact has a CID (Content Identifier) for integrity and deduplication
- **Governance by Default** — Predicates, budgets, and audit trails are first-class primitives
- **Recovery by Default** — Checkpoints at every layer enable rollback and replay
- **Evidence Everywhere** — Cryptographic receipts for all state transitions
- **Protocol Agnostic** — Native CNP, but bridges to A2A, MCP, and external protocols
- **Developer Ergonomics** — Simple Glue facade hides complexity without losing power

1.3 Target Deployment Models

- **Local Node** — Single-machine deployment for development and edge use cases
- **Cluster** — Multi-node deployment with distributed memory and consensus
- **Hosted SaaS** — Fully managed control plane with customer portal and admin panel
- **Hybrid** — Local execution with cloud-based license and billing integration

1.4 Key Terminology

MemPacket

Immutable, content-addressed memory object with CID, content, metadata, and signature

CID Content Identifier in format `cls1-sha256-<hex>`

Agent Autonomous execution unit with DID, memory context, and capability grants

Session Scoped execution context with budget, predicates, and lifecycle

Contract Compiled executable logic with governance, state machine, and resource envelope

CNP Connector Native Protocol — 7-layer communication stack for agent-to-agent messaging

CLS Connector Logic Standard — Contract language, compiler, and runtime

Glue Developer-facing facade providing action-first verbs (start, stop, list, fix, etc.)

Port Typed communication channel with capability gating and UCAN-style attenuation

1.5 Architecture Diagram

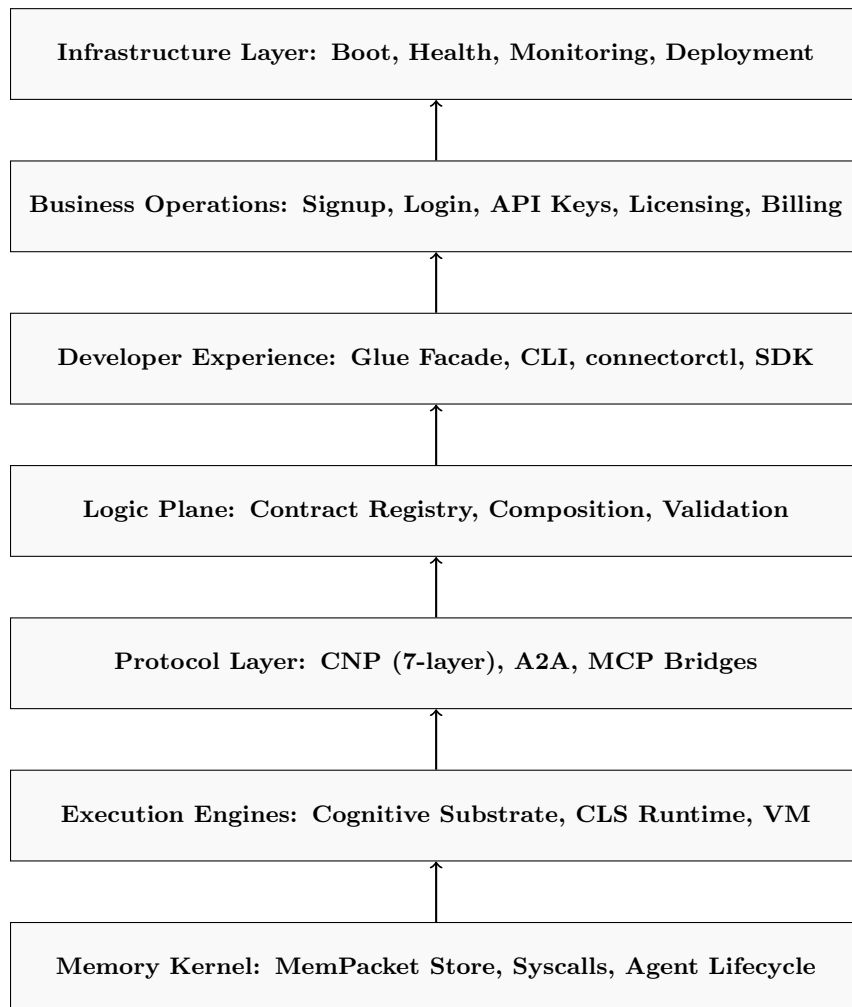


Figure 1: Connector 7-Layer Architecture Stack

1.6 Document Structure

This document is organized as follows:

- **Chapter 2: Memory Kernel** — MemPacket structure, syscall interface, agent lifecycle, session management
- **Chapter 3: Execution Engines** — Cognitive substrate (11-layer thought pipeline), CLS compiler/runtime, contract execution
- **Chapter 4: Protocol Layers** — CNP 7-layer stack, A2A messaging, MCP protocol bridges
- **Chapter 5: Developer Experience** — Glue facade design, CLI grammar, connectorctl commands
- **Chapter 6: Business Operations** — Signup flow, login flow, API key issuance, license activation, billing integration
- **Chapter 7: Node Operations** — 12-stage boot sequence, health probes, agent deployment, monitoring

2 Memory Kernel Layer

2.1 Architecture Overview

The Memory Kernel is the foundation of Connector’s execution model. It provides a content-addressed, append-only storage layer with a syscall interface for agent operations. All agent state, communication, and computation artifacts are stored as **MemPackets** — immutable, cryptographically signed memory objects.

2.2 MemPacket Structure

2.2.1 Core Data Model

```

1 pub struct MemPacket {
2     pub cid: String,           // cls1-sha256-<hex>
3     pub content: MemContent,   // Typed content union
4     pub metadata: MemMetadata, // Timestamps, tags, lineage
5     pub signature: Option<Vec<u8>>, // Ed25519 signature
6     pub encryption: Option<EncryptionEnvelope>,
7 }
8
9 pub enum MemContent {
10     Text(String),
11     Binary(Vec<u8>),
12     Structured(serde_json::Value),
13     Perception(PerceptionObject),
14     Meaning(MeaningObject),
15     Thought(ThoughtRecord),
16     Action(ActionRecord),
17     Evidence(EvidenceChain),
18 }
19
20 pub struct MemMetadata {
21     pub created_at: i64,
22     pub agent_did: Option<String>,
23     pub session_id: Option<String>,
24     pub tags: Vec<String>,
25     pub parent_cids: Vec<String>,

```

```

26 pub lineage: LineageChain,
27 }

```

Listing 1: MemPacket Core Structure

2.2.2 Content Addressing

Every MemPacket is identified by its CID, computed as:

$$\text{CID} = \text{cls1-sha256-}||\text{SHA256}(\text{canonical_json}(\text{content}))$$

This ensures:

- **Deduplication** — Identical content produces identical CIDs
- **Integrity** — Content tampering changes the CID
- **Verifiability** — Any party can recompute and verify the CID
- **Immutability** — MemPackets are write-once, read-many

2.3 Memory Kernel Syscalls

The kernel exposes 60+ syscalls organized into 8 categories:

2.3.1 Memory Operations

```

1 pub enum MemoryKernelOp {
2     // Write operations
3     WriteMemory { content: MemContent, metadata: MemMetadata },
4     WritePerception { perception: PerceptionObject },
5     WriteMeaning { meaning: MeaningObject },
6     WriteThought { thought: ThoughtRecord },
7
8     // Read operations
9     ReadMemory { cid: String },
10    ReadMemoryByTag { tags: Vec<String>, limit: usize },
11    ReadMemoryByAgent { agent_did: String, limit: usize },
12    ReadMemoryBySession { session_id: String },
13
14    // Query operations
15    QueryMemory { predicate: MemPredicate, limit: usize },
16    SearchMemory { query: String, limit: usize },
17
18    // Lineage operations
19    GetLineage { cid: String, depth: usize },
20    GetChildren { parent_cid: String },
21 }

```

Listing 2: Memory Syscalls

2.3.2 Agent Lifecycle Operations

```

1 pub enum AgentLifecycleOp {
2     RegisterAgent {
3         agent_did: String,
4         config: AgentConfig,
5         capabilities: Vec<Capability>
6     },
7     StartAgent { agent_did: String },
8     PauseAgent { agent_did: String },
9     ResumeAgent { agent_did: String },
10    TerminateAgent { agent_did: String },

```

```

11   GetAgentStatus { agent_did: String },
12   ListAgents { filter: AgentFilter },
13 }

```

Listing 3: Agent Lifecycle Syscalls

2.3.3 Session Management Operations

```

1 pub enum SessionOp {
2   CreateSession {
3     agent_did: String,
4     budget: ResourceBudget,
5     predicates: Vec<Predicate>,
6   },
7   CloseSession { session_id: String },
8   GetSessionState { session_id: String },
9   CheckpointSession { session_id: String },
10  RestoreSession { checkpoint_cid: String },
11 }

```

Listing 4: Session Syscalls

2.4 Agent Lifecycle State Machine

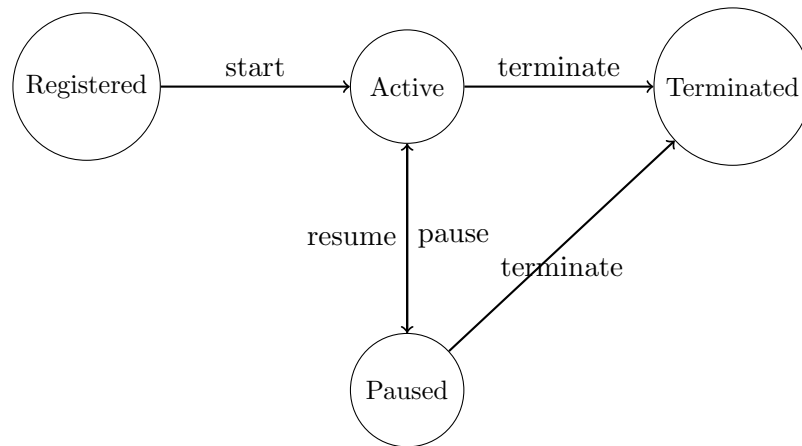


Figure 2: Agent Lifecycle State Transitions

2.5 Session Management

Sessions provide scoped execution contexts with:

- **Resource Budget** — Token limits, memory quotas, time bounds
- **Predicates** — Pre/post conditions, invariants, safety constraints
- **Checkpoint/Restore** — Automatic checkpointing for recovery
- **Isolation** — Session state is isolated from other sessions

2.5.1 Session Lifecycle

1. **Create** — Allocate session with budget and predicates
2. **Execute** — Agent performs operations within session context
3. **Checkpoint** — Periodic state snapshots for recovery
4. **Close** — Finalize session, emit receipt, release resources

2.6 Predicate Enforcement

The kernel enforces an 11-layer predicate chain:

1. **Identity** — Agent DID verification
2. **Capability** — Tool/memory access grants (UCAN-style)
3. **Budget** — Resource consumption limits
4. **Safety** — Invariant checks, bounds validation
5. **Governance** — Pre/post conditions from contracts
6. **Compliance** — Regulatory constraints (HIPAA, GDPR, etc.)
7. **Audit** — Logging requirements
8. **Rate Limit** — Throttling policies
9. **Quota** — Tenant-level limits
10. **Policy** — Organization-specific rules
11. **Emergency** — Circuit breaker, kill switch

2.7 Cryptographic Integrity

2.7.1 Signing

All state transitions are signed with Ed25519:

```

1 pub struct StateTransition {
2     pub from_state: StateCID,
3     pub to_state: StateCID,
4     pub operation: MemoryKernelOp,
5     pub timestamp: i64,
6     pub signature: Vec<u8>, // Ed25519 signature
7 }

```

2.7.2 Execution Receipts

Every session produces a cryptographic receipt:

```

1 pub struct ExecutionReceipt {
2     pub session_id: String,
3     pub agent_did: String,
4     pub operations: Vec<StateTransition>,
5     pub final_state_cid: String,
6     pub budget_consumed: ResourceBudget,
7     pub signature: Vec<u8>,
8 }

```

2.8 Storage Backend

The kernel supports multiple storage backends:

- **SQLite** — Local development, single-node deployments
- **PostgreSQL** — Production deployments with replication

- **S3-compatible** — Object storage for large binary content
- **IPFS** — Decentralized content-addressed storage

All backends implement the same `MemoryStore` trait, ensuring portability.

3 Execution Engines

3.1 Overview

Connector provides two primary execution engines:

1. **Cognitive Substrate** — 11-layer thought pipeline for deliberative reasoning
2. **CLS Runtime** — Contract-based execution with governance and verification

3.2 Cognitive Substrate

The Cognitive Substrate implements a production-grade deliberative reasoning system inspired by cognitive architectures (Soar, ACT-R, BDI) and modern active inference models.

3.2.1 11-Layer Thought Pipeline

1. **Perception** — Raw input → structured `PerceptionObject`
2. **Meaning Extraction** — Perception → `MeaningObject` with semantic content
3. **Tension Detection** — Identify unresolved pressures from meanings
4. **Knowledge Retrieval** — Query memory for relevant context (8 knowledge forms)
5. **Possibility Generation** — Generate typed possibilities from tensions + knowledge
6. **Expertise Application** — Domain-specific evaluation kernels
7. **Evaluation** — Multi-dimensional scoring (feasibility, impact, risk, cost, alignment)
8. **Commitment** — Select and commit to possibilities with revision tracking
9. **Planning** — Build execution DAG with dependencies
10. **Reflection** — Compare expected vs actual outcomes, extract learnings
11. **Exposure** — Multi-audience rendering (human, peer agent, audit, debug)

3.2.2 Core Data Types

```

1 pub struct PerceptionObject {
2     pub source: String,
3     pub modality: Modality, // Text, Vision, Audio, Sensor
4     pub raw_content: Vec<u8>,
5     pub timestamp: i64,
6 }
7
8 pub struct MeaningObject {
9     pub perception_cid: String,
10    pub semantic_content: String,
11    pub entities: Vec<Entity>,
12    pub relations: Vec<Relation>,

```

```

13     pub intent: Option<Intent>,
14 }
15
16 pub struct Tension {
17     pub tension_id: String,
18     pub source_meaning_cid: String,
19     pub description: String,
20     pub intensity: f64, // 0.0 to 1.0
21     pub decay_rate: f64,
22 }
23
24 pub struct Possibility {
25     pub possibility_id: String,
26     pub possibility_type: PossibilityType,
27     pub description: String,
28     pub required_knowledge: Vec<String>,
29     pub estimated_cost: f64,
30 }
31
32 pub enum PossibilityType {
33     Action,
34     Query,
35     Delegation,
36     Composition,
37     Learning,
38     Communication,
39 }

```

Listing 5: Cognitive Substrate Types

3.2.3 Expertise Kernels

Domain-specific evaluation logic is pluggable via the `ExpertiseKernel` trait:

```

1 pub trait ExpertiseKernel: Send + Sync {
2     fn domain(&self) -> &str;
3
4     fn evaluate_possibility(
5         &self,
6         possibility: &Possibility,
7         context: &CognitiveContext,
8     ) -> EvaluationScores;
9
10    fn validate_commitment(
11        &self,
12        commitment: &Commitment,
13        context: &CognitiveContext,
14    ) -> Result<(), String>;
15 }

```

Example domains: General, Medical (HIPAA-aware), Financial (compliance-aware), Legal.

3.2.4 Thought Checkpointing

Every layer produces a checkpoint for recovery:

```

1 pub struct ThoughtCheckpoint {
2     pub checkpoint_id: String,
3     pub layer: CognitiveLayer,
4     pub state: serde_json::Value,
5     pub timestamp: i64,
6     pub parent_checkpoint: Option<String>,
7 }

```

3.3 CLS (Connector Logic Standard)

CLS is a contract language, compiler, and runtime for governed agent logic.

3.3.1 Contract Structure

```

1 pub struct SolutionContract {
2     pub cid: String,           // cls1-sha256-<hex>
3     pub interface: ContractInterface,
4     pub state_machine: StateMachine,
5     pub governance: GovernanceRules,
6     pub resource_envelope: ResourceEnvelope,
7     pub billing_spec: BillingSpec,
8     pub signature: Vec<u8>,
9 }
10
11 pub struct ContractInterface {
12     pub inputs: Vec<InputSpec>,
13     pub outputs: Vec<OutputSpec>,
14     pub tools: Vec<ToolRequirement>,
15     pub memory: Vec<MemoryRequirement>,
16 }
17
18 pub struct GovernanceRules {
19     pub preconditions: Vec<Predicate>,
20     pub postconditions: Vec<Predicate>,
21     pub invariants: Vec<Predicate>,
22 }

```

Listing 6: Solution Contract

3.3.2 Compiler Pipeline

The CLS compiler implements a 6-pass pipeline:

1. **Parse** — YAML surface syntax → AST
2. **Desugar** — Expand syntactic sugar, normalize
3. **Lower** — AST → IR (intermediate representation)
4. **Verify** — Type checking, predicate validation, reachability
5. **Optimize** — Dead code elimination, constant folding, predicate normalization
6. **Emit** — IR → SolutionContract with CID and signature

3.3.3 Runtime Execution

Contract execution follows this lifecycle:

1. **Load** — Retrieve contract by CID
2. **Validate** — Verify signature, check governance predicates
3. **Resolve** — Bind tools, memory, and capabilities
4. **Initialize** — Set up state machine, allocate resources
5. **Execute** — Run state machine with predicate enforcement
6. **Receipt** — Generate signed ExecutionReceipt
7. **Cleanup** — Release resources, persist final state

3.3.4 State Machine Execution

```

1 pub struct StateMachine {
2     pub states: Vec<State>,
3     pub transitions: Vec<Transition>,
4     pub initial_state: String,
5     pub final_states: Vec<String>,
6 }
7
8 pub struct State {
9     pub state_id: String,
10    pub operations: Vec<Operation>,
11    pub predicates: Vec<Predicate>,
12 }
13
14 pub enum Operation {
15     ToolCall { tool_id: String, params: Value },
16     MemoryRead { query: MemPredicate },
17     MemoryWrite { content: MemContent },
18     AgentMessage { target_did: String, payload: Value },
19     Conditional { predicate: Predicate, then_ops: Vec<Operation> },
20 }

```

3.4 Resource Budgets

Both engines enforce resource budgets:

```

1 pub struct ResourceBudget {
2     pub max_tokens: u64,
3     pub max_memory_bytes: u64,
4     pub max_wall_time_ms: u64,
5     pub max_tool_calls: u32,
6     pub max_memory_ops: u32,
7 }
8
9 pub struct ResourceConsumption {
10    pub tokens_used: u64,
11    pub memory_bytes_used: u64,
12    pub wall_time_ms: u64,
13    pub tool_calls: u32,
14    pub memory_ops: u32,
15 }

```

Budget violations trigger immediate session termination with a detailed receipt.

3.5 Integration

The Cognitive Substrate and CLS Runtime are complementary:

- **Cognitive** → **CLS** — Thought pipeline can invoke contracts for structured execution
- **CLS** → **Cognitive** — Contracts can delegate to cognitive substrate for deliberation
- **Hybrid** — Complex agents use both: cognitive for planning, CLS for execution

4 Protocol Layers

4.1 Overview

Connector supports three protocol systems:

1. **CNP (Connector Native Protocol)** — 7-layer stack for agent-to-agent communication

2. **A2A (Agent-to-Agent)** — Cross-platform agent messaging standard
3. **MCP (Model Context Protocol)** — Bridge to external tool/memory providers

4.2 CNP: Connector Native Protocol

CNP is a 7-layer protocol stack designed for secure, typed, capability-gated communication between agents.

4.2.1 Layer Architecture

1. **L1: Content-Addressed Codec** — CID computation, size validation
2. **L2: Encryption** — AEAD encryption with nonce management
3. **L3: Security** — HMAC signatures, anti-replay, TTL enforcement
4. **L4: Port Management** — Typed channels with capability gating
5. **L5: Routing** — Cross-cell routing with delivery guarantees
6. **L6: Session** — Session establishment, lifecycle, teardown
7. **L7: Application** — Payload types and application logic

4.2.2 Message Structure

```

1 pub struct CnpMessage {
2     pub message_id: String,
3     pub sender_pid: ProcessId,
4     pub receiver_pid: ProcessId,
5     pub payload: CnpPayload,
6     pub timestamp: i64,
7 }
8
9 pub enum CnpPayload {
10     Raw(Vec<u8>),
11     Text(String),
12     Sensor(SensorData),
13     Actuation(ActuationCommand),
14     Tensor(TensorData),
15     PacketShare(MemPacketRef),
16     ToolGrant(ToolCapability),
17     Event(EventNotification),
18     Request(RequestPayload),
19     Response(ResponsePayload),
20     PipelineHandoff(PipelineState),
21     Cognitive(CognitiveMessage),
22     Negotiation(NegotiationProposal),
23     KnowledgeRequest(KnowledgeQuery),
24     KnowledgeResponse(KnowledgeData),
25 }

```

Listing 7: CNP Message Types

4.2.3 Port-Based Communication

CNP uses typed ports with capability gating:

```

1 pub struct CnpPort {
2     pub port_id: String,
3     pub port_type: PortType,
4     pub capabilities: Vec<Capability>,
5     pub buffer: VecDeque<CnpSecuredFrame>,
6     pub max_buffer_size: usize,
7 }
8
9 pub enum PortType {
10     Sensor,
11     Actuator,
12     Memory,
13     Tool,
14     Cognitive,
15     Generic,
16 }
17
18 pub struct Capability {
19     pub capability_id: String,
20     pub issuer: String,
21     pub subject: String,
22     pub actions: Vec<String>,
23     pub constraints: Vec<Constraint>,
24     pub expiry: Option<i64>,
25     pub signature: Vec<u8>,
26 }

```

4.2.4 Capability Attenuation

Capabilities can be attenuated (delegated with reduced scope) up to depth 3:

```

1 pub fn attenuate_capability(
2     parent: &Capability,
3     new_subject: String,
4     reduced_actions: Vec<String>,
5     additional_constraints: Vec<Constraint>,
6 ) -> Result<Capability, CnpError> {
7     // Verify parent signature
8     // Check attenuation depth < 3
9     // Verify reduced_actions is subset of parent.actions
10    // Create new capability with parent reference
11    // Sign with current agent's key
12 }

```

4.2.5 Routing and Delivery

CNP provides three delivery guarantees:

- **AtMostOnce** — Fire and forget, no retry
- **AtLeastOnce** — Retry until acknowledged, may duplicate
- **ExactlyOnce** — Idempotent delivery with deduplication

```

1 pub struct CnpRouter {
2     pub routing_table: HashMap<ProcessId, AgentLocation>,
3     pub delivery_receipts: HashMap<String, DeliveryReceipt>,
4     pub retry_queue: VecDeque<RetryEntry>,
5     pub dead_letter_queue: VecDeque<CnpMessage>,
6 }
7

```

```

8 pub struct DeliveryReceipt {
9     pub message_id: String,
10    pub delivered_at: i64,
11    pub signature: Vec<u8>,
12 }

```

4.3 A2A: Agent-to-Agent Protocol

A2A is a cross-platform messaging standard for inter-agent communication.

4.3.1 Message Format

```

1 pub struct A2AMessage {
2     pub from: AgentDID,
3     pub to: AgentDID,
4     pub message_type: A2AMessageType,
5     pub content: serde_json::Value,
6     pub thread_id: Option<String>,
7     pub reply_to: Option<String>,
8 }
9
10 pub enum A2AMessageType {
11     Request,
12     Response,
13     Notification,
14     Handshake,
15     Delegation,
16 }

```

4.3.2 A2A to CNP Bridge

Connector provides a bidirectional bridge:

```

1 pub struct A2ACnpBridge {
2     pub a2a_endpoint: String,
3     pub cnp_router: Arc<CnpRouter>,
4 }
5
6 impl A2ACnpBridge {
7     pub async fn forward_a2a_to_cnp(
8         &self,
9         a2a_msg: A2AMessage,
10    ) -> Result<(), BridgeError> {
11        let cnp_msg = self.convert_a2a_to_cnp(a2a_msg)?;
12        self.cnp_router.route(cnp_msg).await
13    }
14
15    pub async fn forward_cnp_to_a2a(
16        &self,
17        cnp_msg: CnpMessage,
18    ) -> Result<(), BridgeError> {
19        let a2a_msg = self.convert_cnp_to_a2a(cnp_msg)?;
20        self.send_a2a(a2a_msg).await
21    }
22 }

```

4.4 MCP: Model Context Protocol

MCP is an external standard for connecting AI systems to tools and memory providers.

4.4.1 MCP Server Integration

Connector can consume MCP servers as tool providers:

```

1 pub struct McpServerConfig {
2     pub server_name: String,
3     pub command: String,
4     pub args: Vec<String>,
5     pub env: HashMap<String, String>,
6 }
7
8 pub struct McpToolProvider {
9     pub server_config: McpServerConfig,
10    pub available_tools: Vec<McpTool>,
11    pub available_resources: Vec<McpResource>,
12 }
13
14 pub struct McpTool {
15     pub name: String,
16     pub description: String,
17     pub input_schema: serde_json::Value,
18 }

```

4.4.2 Tool Invocation Flow

1. Agent requests tool via Glue or CLS contract
2. Connector checks if tool is MCP-provided
3. If yes, route to MCP bridge
4. MCP bridge invokes external MCP server
5. Response is wrapped in MemPacket and returned
6. All invocations are logged in audit trail

4.5 Protocol Selection Matrix

Use Case	CNP	A2A	MCP
Intra-node agent messaging	X		
Cross-platform agent messaging		X	
External tool integration			X
Capability-gated channels	X		
Typed sensor/actuator data	X		
Cognitive message passing	X	X	
Memory sharing	X	X	X

Table 1: Protocol Selection Guide

4.6 Security Properties

All three protocols provide:

- **Authentication** — Sender identity verification via DIDs and signatures
- **Integrity** — Message tampering detection via HMAC/signatures
- **Confidentiality** — Encryption for sensitive payloads (CNP L2)

- **Non-repudiation** — Cryptographic receipts for all messages
- **Audit Trail** — All protocol operations logged to memory kernel

5 Developer Experience Layer

5.1 Design Philosophy

The Developer Experience (DX) layer is designed around three principles:

1. **Action-First Grammar** — Verbs before nouns (start, stop, list, not agent-start)
2. **Hide Complexity** — Simple surface, powerful primitives underneath
3. **Instant Value** — Useful output from first command, no ceremony

5.2 Glue Facade

Glue is the unified developer surface spanning CLI, API, Rust library, and external language clients.

5.2.1 Core Verbs

Glue provides 12 primary verbs:

Verb	Category	Description
start	Lifecycle	Start an agent or service
stop	Lifecycle	Stop an agent or service
restart	Lifecycle	Restart an agent or service
list	Query	List agents, sessions, memory, etc.
status	Query	Get status of agent, node, or service
inspect	Query	Deep inspection of agent or memory
fix	Maintenance	Auto-repair common issues
logs	Diagnostics	View logs for agent or node
deploy	Operations	Deploy agent or contract
upgrade	Maintenance	Upgrade node or agent
backup	Maintenance	Backup memory or state
restore	Maintenance	Restore from backup

Table 2: Glue Core Verbs

5.2.2 Nouns and Selectors

Glue operates on 10 primary nouns:

- **agent** — Autonomous execution unit
- **session** — Scoped execution context
- **memory** — MemPacket or memory query
- **contract** — CLS solution contract
- **tool** — External capability
- **port** — Communication channel

- **node** — Connector runtime instance
- **cluster** — Multi-node deployment
- **key** — API key or license key
- **user** — Customer account

Selectors use human-readable IDs instead of raw PIDs:

```

1 # Bad (raw PID)
2 connectorctl inspect agent pid:12847
3
4 # Good (readable ID)
5 connectorctl inspect agent medical-assistant-1

```

5.3 connectorctl CLI

The `connectorctl` CLI is the primary interface for node operations.

5.3.1 Command Structure

```

1 connectorctl <verb> <noun> [selector] [flags]

```

Examples:

```

1 # Start an agent
2 connectorctl start agent medical-assistant-1
3
4 # List all running agents
5 connectorctl list agents --status=running
6
7 # Get node status
8 connectorctl status node
9
10 # View agent logs
11 connectorctl logs agent medical-assistant-1 --tail=100
12
13 # Deploy a contract
14 connectorctl deploy contract ./contracts/triage.cls
15
16 # Fix common issues
17 connectorctl fix agent medical-assistant-1
18
19 # Backup memory
20 connectorctl backup memory --output=./backup.tar.gz

```

5.3.2 Output Contracts

All commands produce structured, parseable output:

```

1 # JSON output (default for --json flag)
2 connectorctl status node --json
3 {
4   "node_id": "node-abc123",
5   "status": "healthy",
6   "uptime_seconds": 86400,
7   "agents_running": 5,
8   "memory_used_mb": 1024
9 }
10
11 # Table output (default for interactive)
12 connectorctl list agents

```

ID	STATUS	UPTIME	MEMORY
medical-assistant-1	running	2h 15m	128 MB
data-pipeline-2	running	5h 42m	256 MB
audit-agent-3	paused	1h 03m	64 MB

5.4 Glue API

The Glue API provides HTTP endpoints mirroring CLI verbs:

5.4.1 Endpoint Structure

```
1 POST /api/v1/glue/<verb>/<noun>
```

Examples:

```
1 # Start agent
2 POST /api/v1/glue/start/agent
3 {
4   "agent_id": "medical-assistant-1",
5   "config": { ... }
6 }
7
8 # List agents
9 GET /api/v1/glue/list/agents?status=running
10
11 # Get status
12 GET /api/v1/glue/status/node
13
14 # Deploy contract
15 POST /api/v1/glue/deploy/contract
16 {
17   "contract_cid": "cls1-sha256-abc123...",
18   "config": { ... }
19 }
```

5.4.2 Response Format

All API responses follow this structure:

```
1 {
2   "success": true,
3   "data": { ... },
4   "metadata": {
5     "request_id": "req-xyz789",
6     "timestamp": 1234567890,
7     "duration_ms": 42
8   },
9   "error": null
10 }
```

5.5 Glue Rust Library

For embedded use cases, Glue is available as a Rust library:

```
1 use connector_glue::{Glue, GlueConfig};
2
3 #[tokio::main]
4 async fn main() -> Result<(), Box<dyn std::error::Error>> {
5     let glue = Glue::new(GlueConfig::default()).await?;
6
7     // Start an agent
8     glue.start_agent("medical-assistant-1", None).await?;
```

```

9
10 // List running agents
11 let agents = glue.list_agents(Some("running")).await?;
12
13 // Get node status
14 let status = glue.node_status().await?;
15
16 // Deploy contract
17 let contract_cid = "cls1-sha256-abc123...";
18 glue.deploy_contract(contract_cid, None).await?;
19
20 Ok(())
21 }

```

5.6 Error Handling

Glue provides structured errors with hints:

```

1 {
2   "success": false,
3   "error": {
4     "code": "AGENT_NOT_FOUND",
5     "message": "Agent 'medical-assistant-1' not found",
6     "hint": "Use 'connectorctl list agents' to see available agents",
7     "see_also": [
8       "https://docs.connector.dev/agents/lifecycle"
9     ]
10  }
11 }

```

5.7 Configuration

Glue configuration is hierarchical:

1. **Defaults** — Built-in sensible defaults
2. **Config File** — `./connector/config.toml`
3. **Environment Variables** — `CONNECTOR_*`
4. **CLI Flags** — Command-line overrides

Example config file:

```

1 [node]
2 data_dir = "/var/lib/connector"
3 log_level = "info"
4
5 [api]
6 bind_address = "127.0.0.1:9090"
7 enable_cors = false
8
9 [license]
10 key_file = "/etc/connector/license.key"
11 server_url = "https://license.connector.dev"
12
13 [memory]
14 backend = "postgresql"
15 connection_string = "postgresql://localhost/connector"

```

5.8 Extensibility

Glue supports custom verbs via plugins:

```

1 use connector_glue::{GluePlugin, GlueContext};
2
3 pub struct CustomPlugin;
4
5 impl GluePlugin for CustomPlugin {
6     fn name(&self) -> &str { "custom" }
7
8     fn verbs(&self) -> Vec<&str> {
9         vec!["analyze", "optimize"]
10    }
11
12    async fn execute(
13        &self,
14        verb: &str,
15        noun: &str,
16        ctx: &GlueContext,
17    ) -> Result<serde_json::Value, Box<dyn std::error::Error>> {
18        match verb {
19            "analyze" => self.analyze(noun, ctx).await,
20            "optimize" => self.optimize(noun, ctx).await,
21            _ => Err("Unknown verb".into()),
22        }
23    }
24 }

```

6 Business Operations

6.1 Overview

Connector’s business operations layer handles customer lifecycle, licensing, and billing through a two-server architecture:

1. **Platform Server** — Local node runtime with token-based dashboard access
2. **License/Control Server** — Hosted SaaS with customer portal, admin panel, and billing

6.2 Customer Signup Flow

6.2.1 Step-by-Step Process

1. User visits hosted portal — <https://portal.connector.dev>
2. Click ”Sign Up” — Navigate to signup page
3. Enter details — Name, email, password
4. Submit registration — POST `/api/v1/portal/register`
5. Email verification — Receive verification link
6. Activate account — Click link, account activated
7. Redirect to dashboard — `/app/dashboard`

6.2.2 Registration API

```

1 pub async fn register(
2     State(state): State<SharedState>,
3     Json(req): Json<RegisterRequest>,
4 ) -> Result<Json<RegisterResponse>, ApiError> {
5     // Validate email format
6     validate_email(&req.email)?;
7
8     // Check if user exists
9     if user_exists(&state.db, &req.email).await? {
10         return Err(ApiError::UserAlreadyExists);
11     }
12
13     // Hash password with Argon2
14     let password_hash = hash_password(&req.password)?;
15
16     // Create user record
17     let user_id = create_user(&state.db, CreateUserParams {
18         email: req.email,
19         name: req.name,
20         password_hash,
21     }).await?;
22
23     // Send verification email
24     send_verification_email(&state.email_client, &user_id).await?;
25
26     Ok(Json(RegisterResponse {
27         user_id,
28         message: "Verification email sent",
29     })))
30 }

```

Listing 8: Registration Handler

6.2.3 Database Schema

```

1 CREATE TABLE users (
2     user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     email VARCHAR(255) UNIQUE NOT NULL,
4     name VARCHAR(255) NOT NULL,
5     password_hash VARCHAR(255) NOT NULL,
6     email_verified BOOLEAN DEFAULT FALSE,
7     created_at TIMESTAMPTZ DEFAULT NOW(),
8     updated_at TIMESTAMPTZ DEFAULT NOW()
9 );
10
11 CREATE TABLE email_verifications (
12     verification_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
13     user_id UUID REFERENCES users(user_id),
14     token VARCHAR(64) UNIQUE NOT NULL,
15     expires_at TIMESTAMPTZ NOT NULL,
16     verified_at TIMESTAMPTZ
17 );

```

6.3 Login Flow

6.3.1 Portal Login (Email/Password)

1. User visits login page — /login
2. Enter credentials — Email and password
3. Submit — POST /api/v1/portal/login

4. **Verify credentials** — Check password hash
5. **Check TOTP (if enabled)** — Prompt for 2FA code
6. **Issue session token** — JWT with 24h expiry
7. **Redirect to app** — /app/dashboard

6.3.2 Login API

```

1 pub async fn login(
2   State(state): State<SharedState>,
3   Json(req): Json<LoginRequest>,
4 ) -> Result<Json<LoginResponse>, ApiError> {
5   // Fetch user by email
6   let user = get_user_by_email(&state.db, &req.email).await?
7     .ok_or(ApiError::InvalidCredentials)?;
8
9   // Verify password
10  verify_password(&req.password, &user.password_hash)?;
11
12  // Check if TOTP is enabled
13  if user.totp_enabled {
14    if let Some(totp_code) = req.totp_code {
15      verify_totp(&user.totp_secret, &totp_code)?;
16    } else {
17      return Ok(Json(LoginResponse::TotpRequired));
18    }
19  }
20
21  // Generate session token (JWT)
22  let token = generate_jwt(&state.jwt_secret, &user.user_id)?;
23
24  Ok(Json(LoginResponse::Success {
25    token,
26    user: PortalUser {
27      user_id: user.user_id,
28      email: user.email,
29      name: user.name,
30    },
31  })))
32 }

```

6.3.3 Dashboard Login (Token-Only)

The local node dashboard uses API keys/tokens only:

1. **User visits dashboard** — <http://localhost:9090>
2. **Enter API key** — Paste key from portal
3. **Validate key** — Check against local config or license server
4. **Grant access** — Store key in localStorage

No email/password login is offered on the local dashboard.

6.4 API Key Management

6.4.1 Key Issuance Flow

1. **User logs into portal** — Authenticated session

2. **Navigate to API Keys** — /app/apikeys
3. **Click "Create Key"** — Modal opens
4. **Enter key name** — Descriptive label
5. **Submit** — POST /api/v1/portal/api-keys
6. **Receive key** — Display once, copy to clipboard
7. **Store securely** — User saves key (not shown again)

6.4.2 API Key Structure

```

1 pub struct ApiKey {
2     pub key_id: String,
3     pub user_id: String,
4     pub key_hash: String,          // SHA-256 hash
5     pub key_prefix: String,       // First 8 chars for display
6     pub name: String,
7     pub scopes: Vec<String>,
8     pub created_at: i64,
9     pub last_used_at: Option<i64>,
10    pub revoked_at: Option<i64>,
11 }
12
13 // Key format: ck_live_<32-char-random>
14 // Example: ck_live_a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6

```

6.4.3 Key Validation

```

1 pub async fn validate_api_key(
2     db: &PgPool,
3     key: &str,
4 ) -> Result<ApiKey, ApiError> {
5     // Hash the provided key
6     let key_hash = sha256(key);
7
8     // Look up in database
9     let api_key = sqlx::query_as::<_, ApiKey>(
10         "SELECT * FROM api_keys WHERE key_hash = $1 AND revoked_at IS NULL"
11     )
12     .bind(&key_hash)
13     .fetch_optional(db)
14     .await?
15     .ok_or(ApiError::InvalidApiKey)?;
16
17     // Update last_used_at
18     update_last_used(db, &api_key.key_id).await?;
19
20     Ok(api_key)
21 }

```

6.5 License Activation

6.5.1 Activation Flow

1. **User obtains license key** — From portal after purchase
2. **Install Connector node** — Download and install binary
3. **Run activation command** — `connectorctl activate <key>`

4. **Contact license server** — POST /api/v1/license/activate
5. **Verify key validity** — Check tier, expiry, instance limit
6. **Generate machine fingerprint** — CPU, MAC, hostname
7. **Record activation** — Store instance in database
8. **Return license file** — Signed license with entitlements
9. **Save locally** — /etc/connector/license.key
10. **Node boots** — Reads license, enforces limits

6.5.2 License Structure

```

1 pub struct License {
2     pub license_id: String,
3     pub customer_id: String,
4     pub tier: LicenseTier,
5     pub agent_limit: u32,
6     pub issued_at: i64,
7     pub expires_at: Option<i64>,
8     pub features: Vec<String>,
9     pub signature: Vec<u8>, // Ed25519 signature
10 }
11
12 pub enum LicenseTier {
13     Free,
14     Starter,
15     Professional,
16     Enterprise,
17 }

```

6.5.3 Activation API

```

1 pub async fn activate_license(
2     State(state): State<SharedState>,
3     Json(req): Json<ActivateRequest>,
4 ) -> Result<Json<ActivateResponse>, ApiError> {
5     // Validate license key
6     let key = get_license_key(&state.db, &req.key).await?
7         .ok_or(ApiError::InvalidLicenseKey)?;
8
9     // Check if key is already activated on max instances
10    let instance_count = count_active_instances(&state.db, &key.key_id).await?;
11    if instance_count >= key.max_instances {
12        return Err(ApiError::InstanceLimitReached);
13    }
14
15    // Generate machine fingerprint
16    let machine_id = compute_machine_id(&req.machine_info)?;
17
18    // Record activation
19    let instance_id = create_instance(&state.db, CreateInstanceParams {
20        key_id: key.key_id.clone(),
21        machine_id,
22        hostname: req.hostname,
23        activated_at: now(),
24    }).await?;
25
26    // Generate signed license file

```

```

27     let license = License {
28         license_id: key.key_id,
29         customer_id: key.customer_id,
30         tier: key.tier,
31         agent_limit: key.agent_limit,
32         issued_at: now(),
33         expires_at: key.expires_at,
34         features: key.features,
35         signature: sign_license(&state.signing_key, &key)?,
36     };
37
38     Ok(Json(ActivateResponse {
39         instance_id,
40         license,
41     })))
42 }

```

6.6 Billing Integration

6.6.1 Stripe Integration

Connector uses Stripe for payment processing:

1. **User selects plan** — Free, Starter, Pro, Enterprise
2. **Click "Subscribe"** — Redirect to Stripe Checkout
3. **Enter payment details** — Stripe-hosted form
4. **Confirm subscription** — Stripe processes payment
5. **Webhook notification** — Stripe → Connector webhook
6. **Update entitlements** — Increase agent limit, enable features
7. **Issue license key** — Generate and display key

6.6.2 Webhook Handler

```

1 pub async fn stripe_webhook(
2     State(state): State<SharedState>,
3     headers: HeaderMap,
4     body: String,
5 ) -> Result<StatusCode, ApiError> {
6     // Verify webhook signature
7     let signature = headers.get("stripe-signature")
8         .ok_or(ApiError::MissingSignature)?;
9     verify_stripe_signature(&state.stripe_secret, &body, signature)?;
10
11     // Parse event
12     let event: StripeEvent = serde_json::from_str(&body)?;
13
14     match event.event_type.as_str() {
15         "customer.subscription.created" => {
16             handle_subscription_created(&state.db, event.data).await?;
17         }
18         "customer.subscription.updated" => {
19             handle_subscription_updated(&state.db, event.data).await?;
20         }
21         "customer.subscription.deleted" => {
22             handle_subscription_deleted(&state.db, event.data).await?;
23     }

```

```

24     "invoice.payment_failed" => {
25         handle_payment_failed(&state.db, event.data).await?;
26     }
27     _ => {}
28 }
29
30 Ok(StatusCode::OK)
31 }

```

6.7 Tracking and Analytics

6.7.1 Usage Tracking

The platform server reports usage metrics to the license server:

```

1 pub async fn report_usage(
2     license_server_url: &str,
3     instance_id: &str,
4     metrics: UsageMetrics,
5 ) -> Result<(), ReportError> {
6     let client = reqwest::Client::new();
7     client.post(format!("{}/api/v1/usage/report", license_server_url))
8         .json(&UsageReport {
9             instance_id: instance_id.to_string(),
10            timestamp: now(),
11            agents_running: metrics.agents_running,
12            memory_packets: metrics.memory_packets,
13            tokens_consumed: metrics.tokens_consumed,
14            uptime_seconds: metrics.uptime_seconds,
15        })
16        .send()
17        .await?;
18     Ok(())
19 }

```

6.7.2 Metrics Collected

- **Agent count** — Number of active agents
- **Memory usage** — MemPacket count and total bytes
- **Token consumption** — LLM tokens used
- **Uptime** — Node uptime in seconds
- **Session count** — Active sessions
- **Tool invocations** — External tool calls

These metrics feed into:

- **Customer dashboard** — Usage graphs and alerts
- **Admin panel** — Fleet-wide analytics
- **Billing** — Usage-based pricing (future)

7 Node Operations

7.1 Overview

Connector nodes are designed for infrastructure-grade deployment with production-ready boot sequences, health monitoring, and operational tooling.

7.2 12-Stage Boot Sequence

The node boot process follows a rigorous 12-stage pipeline:

1. **IDENTITY** — Load node identity, generate DID if needed
2. **CONFIG** — Parse configuration from file, env vars, and CLI flags
3. **SECRETS** — Load encryption keys, signing keys, API tokens
4. **STORAGE** — Initialize memory kernel storage backend
5. **KERNEL** — Boot memory kernel, verify integrity
6. **POLICIES** — Load governance policies and predicates
7. **SCHEDULER** — Initialize agent scheduler and resource manager
8. **CAPABILITIES** — Register available tools and capabilities
9. **RESTORE** — Restore agents and sessions from checkpoints
10. **SERVICES** — Start HTTP API, WebSocket, and protocol bridges
11. **ACCESS** — Enable dashboard and external access
12. **READY** — Signal readiness to systemd/Kubernetes

7.2.1 Boot Reporter

Each stage reports progress:

```

1 pub struct BootReporter {
2     pub current_stage: BootStage,
3     pub total_stages: usize,
4     pub stage_start: Instant,
5     pub boot_start: Instant,
6 }
7
8 impl BootReporter {
9     pub fn report_stage(&self, stage: BootStage, status: StageStatus) {
10         let elapsed = self.stage_start.elapsed();
11         match status {
12             StageStatus::Starting => {
13                 println!("[{:2}/{}] {} ...",
14                     stage as usize, self.total_stages, stage.name());
15             }
16             StageStatus::Complete => {
17                 println!("[{:2}/{}] {} OK ({:.2}s)",
18                     stage as usize, self.total_stages,
19                     stage.name(), elapsed.as_secs_f64());
20             }
21             StageStatus::Failed(err) => {
22                 eprintln!("[{:2}/{}] {} FAILED {}",
23                     stage as usize, self.total_stages,
24                     stage.name(), err);
25             }
26         }
27     }
28 }

```

7.2.2 systemd Integration

Connector integrates with systemd for production deployments:

```

1 [Unit]
2 Description=Connector Agentic Runtime
3 After=network.target postgresql.service
4 Requires=postgresql.service
5
6 [Service]
7 Type=notify
8 User=connector
9 Group=connector
10 ExecStart=/usr/local/bin/connector-node \
11     --config /etc/connector/config.toml
12 Restart=on-failure
13 RestartSec=10s
14 WatchdogSec=60s
15 NotifyAccess=main
16
17 [Install]
18 WantedBy=multi-user.target

```

Listing 9: /etc/systemd/system/connector-node.service

The node sends `sd_notify` signals:

```

1 use sd_notify::NotifyState;
2
3 // During boot
4 sd_notify::notify(false, &[NotifyState::Status("Booting...")]).ok();
5
6 // When ready
7 sd_notify::notify(false, &[NotifyState::Ready]).ok();
8
9 // Watchdog heartbeat
10 sd_notify::notify(false, &[NotifyState::Watchdog]).ok();

```

7.3 Health Probes

Connector provides three health probe endpoints for Kubernetes and load balancers:

7.3.1 Liveness Probe

Checks if the node is alive (not deadlocked):

```

1 // GET /health/live
2 pub async fn liveness_probe(
3     State(state): State<SharedState>,
4 ) -> Result<Json<LivenessResponse>, StatusCode> {
5     // Simple heartbeat check
6     if state.is_alive() {
7         Ok(Json(LivenessResponse { status: "alive" }))
8     } else {
9         Err(StatusCode::SERVICE_UNAVAILABLE)
10    }
11 }

```

7.3.2 Readiness Probe

Checks if the node is ready to accept traffic:

```

1 // GET /health/ready
2 pub async fn readiness_probe(
3     State(state): State<SharedState>,
4 ) -> Result<Json<ReadinessResponse>, StatusCode> {
5     // Check all critical services
6     let checks = vec![
7         ("memory_kernel", state.memory_kernel.is_ready()),
8         ("scheduler", state.scheduler.is_ready()),
9         ("storage", state.storage.is_ready()),
10    ];
11
12    let all_ready = checks.iter().all(|(_, ready)| *ready);
13
14    if all_ready {
15        Ok(Json(ReadinessResponse {
16            status: "ready",
17            checks: checks.into_iter()
18                .map(|(name, ready)| (name.to_string(), ready))
19                .collect(),
20        }))
21    } else {
22        Err(StatusCode::SERVICE_UNAVAILABLE)
23    }
24 }

```

7.3.3 Startup Probe

Checks if the node has completed boot:

```

1 // GET /health/startup
2 pub async fn startup_probe(
3     State(state): State<SharedState>,
4 ) -> Result<Json<StartupResponse>, StatusCode> {
5     if state.boot_complete {
6         Ok(Json(StartupResponse {
7             status: "started",
8             boot_duration_ms: state.boot_duration.as_millis() as u64,
9         }))
10    } else {
11        Err(StatusCode::SERVICE_UNAVAILABLE)
12    }
13 }

```

7.4 Agent Deployment

7.4.1 Deployment Methods

Agents can be deployed via:

1. **CLI** — `connectorctl deploy agent ./agent.toml`
2. **API** — `POST /api/v1/agents`
3. **Contract** — CLS contract with agent definition
4. **Git** — Pull from repository and auto-deploy

7.4.2 Agent Manifest

```

1 [agent]
2 name = "medical-assistant-1"
3 description = "HIPAA-compliant medical triage agent"
4 version = "1.0.0"
5
6 [runtime]
7 model = "gpt-4"
8 temperature = 0.7
9 max_tokens = 2000
10
11 [capabilities]
12 tools = ["search_medical_db", "schedule_appointment"]
13 memory_access = ["patient_records", "medical_knowledge"]
14
15 [governance]
16 predicates = ["hipaa_compliance", "patient_consent"]
17 budget.max_tokens = 100000
18 budget.max_wall_time_ms = 300000
19
20 [deployment]
21 replicas = 2
22 auto_restart = true
23 health_check_interval_ms = 30000

```

Listing 10: agent.toml

7.4.3 Deployment Flow

1. **Parse manifest** — Validate TOML structure
2. **Verify capabilities** — Check tool/memory availability
3. **Compile governance** — Convert predicates to executable form
4. **Allocate resources** — Reserve memory and compute
5. **Register agent** — Create agent record in kernel
6. **Start agent** — Transition to Active state
7. **Health check** — Verify agent is responsive
8. **Report success** — Return agent DID and status

7.5 Monitoring and Observability

7.5.1 Metrics Export

Connector exports Prometheus-compatible metrics:

```

1 // GET /metrics
2 pub async fn metrics(
3     State(state): State<SharedState>,
4 ) -> String {
5     format!(
6         "# HELP connector_agents_running Number of running agents\n\
7         # TYPE connector_agents_running gauge\n\
8         connector_agents_running {}\n\
9         \n\
10        # HELP connector_memory_packets Total memory packets\n\
11        # TYPE connector_memory_packets counter\n\
12        connector_memory_packets {}\n\
13        \n\
14        # HELP connector_tokens_consumed Total tokens consumed\n\

```

```

15         # TYPE connector_tokens_consumed counter\n\
16         connector_tokens_consumed {}\n",
17         state.metrics.agents_running,
18         state.metrics.memory_packets,
19         state.metrics.tokens_consumed,
20     )
21 }

```

7.5.2 Structured Logging

All logs use structured JSON format:

```

1 use tracing::{info, error};
2
3 info!(
4     agent_id = %agent.did,
5     session_id = %session.id,
6     operation = "start",
7     "Agent started successfully"
8 );
9
10 error!(
11     agent_id = %agent.did,
12     error = %err,
13     "Agent failed to start"
14 );

```

7.5.3 Distributed Tracing

Connector supports OpenTelemetry for distributed tracing:

```

1 use opentelemetry::trace::{Tracer, Span};
2
3 let tracer = global::tracer("connector");
4 let mut span = tracer.start("agent.execute");
5 span.set_attribute(KeyValue::new("agent.did", agent.did.clone()));
6
7 // ... execute agent ...
8
9 span.end();

```

7.6 Graceful Shutdown

7.6.1 Shutdown Sequence

1. **Receive signal** — SIGTERM or SIGINT
2. **Stop accepting new requests** — Return 503 from health probes
3. **Drain connections** — Wait for in-flight requests (max 30s)
4. **Pause agents** — Checkpoint all running agents
5. **Close sessions** — Finalize sessions, emit receipts
6. **Flush memory** — Persist pending MemPackets
7. **Shutdown services** — Close HTTP server, WebSocket, etc.
8. **Exit** — Return exit code 0

```

1 pub async fn graceful_shutdown(
2     state: SharedState,
3     shutdown_rx: tokio::sync::watch::Receiver<bool>,
4 ) {
5     shutdown_rx.changed().await.ok();
6
7     info!("Graceful shutdown initiated");
8
9     // Stop accepting new requests
10    state.set_draining(true);
11
12    // Wait for in-flight requests
13    tokio::time::sleep(Duration::from_secs(5)).await;
14
15    // Checkpoint agents
16    for agent in state.agents.iter() {
17        agent.checkpoint().await.ok();
18    }
19
20    // Close sessions
21    for session in state.sessions.iter() {
22        session.close().await.ok();
23    }
24
25    // Flush memory
26    state.memory_kernel.flush().await.ok();
27
28    info!("Graceful shutdown complete");
29 }

```

7.7 Backup and Restore

7.7.1 Backup

```

1 # Full backup
2 connectorctl backup memory --output=/backups/connector-$(date +%Y%m%d).tar.gz
3
4 # Incremental backup (since last checkpoint)
5 connectorctl backup memory --incremental --output=/backups/incremental.tar.gz
6
7 # Backup specific agent
8 connectorctl backup agent medical-assistant-1 --output=/backups/agent.tar.gz

```

7.7.2 Restore

```

1 # Restore from backup
2 connectorctl restore memory --input=/backups/connector-20260322.tar.gz
3
4 # Restore specific agent
5 connectorctl restore agent --input=/backups/agent.tar.gz

```

7.8 Upgrade Process

7.8.1 Rolling Upgrade

For cluster deployments:

1. **Download new version** — `connectorctl upgrade --version=1.2.0`
2. **Verify checksum** — Validate binary integrity
3. **Backup current state** — Automatic checkpoint

4. **Stop node** — Graceful shutdown
5. **Replace binary** — Swap old binary with new
6. **Start node** — Boot with new version
7. **Verify health** — Check all probes pass
8. **Restore agents** — Resume from checkpoints

7.8.2 Rollback

If upgrade fails:

```

1 # Automatic rollback on failure
2 connectorctl upgrade --version=1.2.0 --rollback-on-failure
3
4 # Manual rollback
5 connectorctl rollback --to-version=1.1.0

```

7.9 Disaster Recovery

7.9.1 Recovery Scenarios

Scenario	Recovery Procedure
Node crash	Restart node, agents auto-restore from last checkpoint
Disk corruption	Restore from backup, replay audit log
Database failure	Switch to replica, restore from WAL
Network partition	Agents pause, resume when connectivity restored
License expiry	Agents pause, resume after license renewal

Table 3: Disaster Recovery Procedures

7.9.2 Recovery Time Objectives

- **RTO (Recovery Time Objective)** — 5 minutes for single-node, 30 seconds for cluster
- **RPO (Recovery Point Objective)** — Last checkpoint (typically ; 1 minute data loss)